

Print all possible combinations from the given string.

ANSWER IN C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    scanf("%s", str);
```

```
    int len = strlen(str);
```

```
    int n = 1 << len; // total combinations
```

```
    for(int i = 1; i < n; i++) {
```

```
        for(int j = 0; j < len; j++) {
```

```
if(i & (1 << j))  
    printf("%c", str[j]);  
}  
printf("\n");  
}  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;

public class StringCombinations {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String str = sc.next();

        int len = str.length();

        int n = 1 << len;

        for(int i = 1; i < n; i++) {

            for(int j = 0; j < len; j++) {
```

```
if((i & (1 << j)) > 0)
    System.out.print(str.charAt(j));
}
System.out.println();
}
}
}
```

2) Given a sliding window of size k print the maximum of the numbers under the sliding window. Example: Consider a sliding window of size k equals 3. Let the array be $[3, 2, 7, 6, 5, 1, 2, 3, 4]$ the output should print 7 as the first output as first window contains $\{3, 2, 7\}$ and second window contains $\{2, 7, 6\}$ and so on and the final output is $\{7, 7, 7, 6, 5, 3, 4\}$

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {  
    int n, k;  
    scanf("%d", &n);  
    int arr[n];  
    for(int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);  
    scanf("%d", &k);  
  
    for(int i = 0; i <= n - k; i++) {  
        int max = arr[i];
```

```
for(int j = i + 1; j < i + k; j++) {  
    if(arr[j] > max)  
        max = arr[j];  
}  
printf("%d ", max);  
}  
return 0;  
}
```


ANSWER IN JAVA

```
import java.util.*;

public class SlidingWindowMax {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        int k = sc.nextInt();
        for(int i = 0; i <= n - k; i++) {
            int max = arr[i];
```

```
for(int j = i + 1; j < i + k; j++) {  
    if(arr[j] > max)  
        max = arr[j];  
}  
System.out.print(max + " ");  
}  
}  
}
```

3)Write a program to print the below pattern

For $n = 6$,

print:

1 7 12 16 19 21

2 8 13 17 20

3 9 14 18

4 10 15

5 11

6

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    for(int i = 1; i <= n; i++) {
        int num = i;
        int diff = n - 1;
        while(num <= n*(n+1)/2) {
            printf("%d ", num);
            num += diff;
            diff--;
        }
        printf("\n");
    }
    return 0;
}
```

ANSWER IN JAVA

```
import java.util.*;

public class PatternPrint {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for(int i = 1; i <= n; i++) {

            int num = i;

            int diff = n - 1;

            while(diff >= 0) {

                System.out.print(num + " ");

                num += diff;

                diff--;

            }    System.out.println();

        } } }
```

4) Given a bigger $N \times N$ matrix and a smaller $M \times M$ matrix, print TRUE if the smaller matrix can be found in the bigger matrix, else print FALSE.

ANSWER IN C

```
#include <stdio.h>

int main() {
    int N, M, i, j, x, y, found = 0;
    scanf("%d", &N);
    int big[N][N];
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            scanf("%d", &big[i][j]);

    scanf("%d", &M);
    int small[M][M];
    for(i=0; i<M; i++)
        for(j=0; j<M; j++)
            scanf("%d", &small[i][j]);
```

```
for(i=0; i<=N-M; i++) {  
    for(j=0; j<=N-M; j++) {  
        int match = 1;  
        for(x=0; x<M; x++) {  
            for(y=0; y<M; y++) {  
                if(big[i+x][j+y] != small[x][y]) {  
                    match = 0;  
                    break;  
                }  
            }  
            if(!match) break;  
        }  
        if(match) {  
            found = 1;  
            break;  
        }  
    }  
}
```



```
if(found) break;  
}
```

```
if(found)  
    printf("TRUE\n");  
else  
    printf("FALSE\n");
```

```
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;

public class MatrixSearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        int[][] big = new int[N][N];

        for(int i=0; i<N; i++)

            for(int j=0; j<N; j++)

                big[i][j] = sc.nextInt();

        int M = sc.nextInt();

        int[][] small = new int[M][M];

        for(int i=0; i<M; i++)

            for(int j=0; j<M; j++)

                small[i][j] = sc.nextInt();

    }

}
```

```
boolean found = false;
for(int i=0; i<=N-M; i++) {
    for(int j=0; j<=N-M; j++) {
        boolean match = true;
        for(int x=0; x<M; x++) {
            for(int y=0; y<M; y++) {
                if(big[i+x][j+y] != small[x][y]) {
                    match = false;
                    break;
                }
            }
            if(!match) break;
        }
        if(match) {
            found = true;
            break;
        }
    }
}
```

```
if(found) break;
```

```
}
```

```
if(found)
```

```
    System.out.println("TRUE");
```

```
else
```

```
    System.out.println("FALSE");
```

```
}
```

```
}
```

5) Given two matrices a and b , both of size $N \times N$, determine if matrix a can be transformed to matrix b by rotating it 90° , 180° , or 270° . If so, print TRUE, else print FALSE.

ANSWER IN C

```
#include <stdio.h>

int N;

int check(int a[N][N], int b[N][N]) {
    int i, j, rot90=1, rot180=1, rot270=1;
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            if(a[N-j-1][i] != b[i][j])
                rot90 = 0;
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            if(a[N-i-1][N-j-1] != b[i][j])
                rot180 = 0;
```

```
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            if(a[j][N-i-1] != b[i][j])
                rot270 = 0;
    return rot90 || rot180 || rot270;
}

int main() {
    scanf("%d", &N);
    int a[N][N], b[N][N], i, j;
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            scanf("%d", &a[i][j]);
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            scanf("%d", &b[i][j]);
```

```
if(check(a,b))  
    printf("TRUE\n");  
else  
    printf("FALSE\n");  
  
return 0;  
}
```


ANSWER IN JAVA

```
import java.util.*;
public class MatrixRotationCheck {
    static boolean check(int[][] a, int[][] b, int N) {
        boolean rot90 = true, rot180 = true, rot270 = true;
        for(int i=0; i<N; i++)
            for(int j=0; j<N; j++)
                if(a[N-j-1][i] != b[i][j])
                    rot90 = false;
        for(int i=0; i<N; i++)
            for(int j=0; j<N; j++)
                if(a[N-i-1][N-j-1] != b[i][j])
                    rot180 = false;
```

```
for(int i=0; i<N; i++)
    for(int j=0; j<N; j++)
        if(a[j][N-i-1] != b[i][j])
            rot270 = false;

return rot90 || rot180 || rot270;
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    int[][] a = new int[N][N];
    int[][] b = new int[N][N];

    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            a[i][j] = sc.nextInt();
}
```

```
for(int i=0; i<N; i++)  
    for(int j=0; j<N; j++)  
        b[i][j] = sc.nextInt();  
  
if(check(a,b,N))  
    System.out.println("TRUE");  
else  
    System.out.println("FALSE");  
}  
}
```

6) In addition to the above question, you have to check if matrix a can be transformed to matrix b by **mirroring vertically or horizontally**.

ANSWER IN C

```
#include <stdio.h>

int N;

void readMatrix(int mat[N][N]) {
    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            scanf("%d", &mat[i][j]);
}

int isEqual(int a[N][N], int b[N][N]) {
    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            if(a[i][j] != b[i][j])
                return 0;
    return 1;
}
```

```
void rotate90(int src[N][N], int dest[N][N]) {  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[j][N-1-i] = src[i][j];  
}  
void mirrorHorizontal(int src[N][N], int dest[N][N]) {  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[N-1-i][j] = src[i][j];  
}  
void mirrorVertical(int src[N][N], int dest[N][N]) {  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[i][N-1-j] = src[i][j];  
}
```

```
int main() {  
    scanf("%d", &N);  
    int a[N][N], b[N][N], temp[N][N];  
    readMatrix(a);  
    readMatrix(b);  
    if(isEqual(a,b)) {  
        printf("TRUE\n");  
        return 0;  
    }  
    rotate90(a,temp);  
    if(isEqual(temp,b)) {  
        printf("TRUE\n");  
        return 0;  
    }  
    rotate90(temp,temp);  
    if(isEqual(temp,b)) { printf("TRUE\n"); return 0; }
```

```
rotate90(temp,temp);  
    if(isEqual(temp,b)) {  
        printf("TRUE\n");  
        return 0;  
    }  
    mirrorHorizontal(a,temp);  
    if(isEqual(temp,b)) {  
        printf("TRUE\n");  
        return 0;  
    }  
    mirrorVertical(a,temp);  
    if(isEqual(temp,b)) {  
        printf("TRUE\n");  
        return 0;  
    }  
    printf("FALSE\n"); return 0; }
```



```
boolean found = false;
```

```
    for(int i=0; i<=N-M; i++) {  
        for(int j=0; j<=N-M; j++) {  
            boolean match = true;  
            for(int x=0; x<M; x++)  
                for(int y=0; y<M; y++)  
                    if(big[i+x][j+y] != small[x][y])  
                        match = false;  
            if(match) {  
                found = true;  
                break;  
            }  
        }  
        if(found) break;  
    }
```



```
static int[][] rotate90(int[][] src) {  
    int[][] dest = new int[N][N];  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[j][N-1-i] = src[i][j];  
    return dest;  
}
```

```
static int[][] mirrorHorizontal(int[][] src) {  
    int[][] dest = new int[N][N];  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[N-1-i][j] = src[i][j];  
    return dest;  
}
```

```
static int[][] mirrorVertical(int[][] src) {  
    int[][] dest = new int[N][N];  
    for(int i=0; i<N; i++)  
        for(int j=0; j<N; j++)  
            dest[i][N-1-j] = src[i][j];  
    return dest;  
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    N = sc.nextInt();  
    int[][] a = new int[N][N];  
    int[][] b = new int[N][N];  
    readMatrix(sc,a);  
    readMatrix(sc,b);  
  
    if(isEqual(a,b)) {  
        System.out.println("TRUE");  
        return;  
    }  
}
```

```
int[][] temp = rotate90(a);
    if(isEqual(temp,b)) {
        System.out.println("TRUE");
        return;
    }
    temp = rotate90(temp);
    if(isEqual(temp,b)) {
        System.out.println("TRUE");
        return;
    }
    temp = rotate90(temp);
    if(isEqual(temp,b)) {
        System.out.println("TRUE");
        return;
    }
}
```

```
temp = mirrorHorizontal(a);  
    if(isEqual(temp,b)) {  
        System.out.println("TRUE");  
        return;  
    }  
    temp = mirrorVertical(a);  
    if(isEqual(temp,b)) {  
        System.out.println("TRUE");  
        return;  
    }  
  
    System.out.println("FALSE");  
}  
}
```

43) Given an array of integers, rearrange the array in such a way that:

- The first element is the **maximum**,
- The second element is the **minimum**,
- The third is the **second maximum**,
- The fourth is the **second minimum**, and so on.

Example:

Input : {1, 2, 3, 4, 5, 6, 7} Output : {7, 1, 6, 2, 5, 3, 4}

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n], res[n];
```

```
    for(int i=0;i<n;i++)
```

```
        scanf("%d",&arr[i]);
```

```
    int start=0, end=n-1, idx=0;
```

```
    while(start <= end) {
```

```
        if(idx%2==0)
```



```
res[idx++] = arr[end--];  
    else  
        res[idx++] = arr[start++];  
}
```

```
for(int i=0;i<n;i++)  
    printf("%d ",res[i]);  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;

public class MaxMinRearrange {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();
        int res[] = new int[n];
        int start=0, end=n-1, idx=0;
```

```
while(start <= end) {  
    if(idx%2==0)  
        res[idx++] = arr[end--];  
    else  
        res[idx++] = arr[start++];  
}  
  
for(int i=0;i<n;i++)  
    System.out.print(res[i]+" ");  
}  
}
```

44) You're given a number n . If we write all the numbers from 1 to n on paper, we have to find the number of characters written on the paper.

For example:

- If $n = 13$, the output should be 18
- If $n = 101$, the output should be 195

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, i, count = 0;
    printf("Enter n: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++) {
        int num = i;
        while(num != 0) {
            count++;
            num /= 10;
        }
    }
    printf("Total digits used: %d\n", count);
    return 0;
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class NumberCharacters {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        int count = 0;

        for(int i = 1; i <= n; i++) {
            int num = i;
            while(num > 0) {
```

```
count++;
```

```
    num /= 10;
```

```
    }
```

```
}
```

```
System.out.println("Total digits used: " + count);
```

```
}
```

```
}
```

45) A number is called a **binary-decimal** if all the digits in the number are either 1 or 0.

Any number can be written as a sum of binary-decimals.

Your task is to find the **minimum number** of binary-decimals needed to represent a number.

Example:

- Input: 32 $\rightarrow$ Output: 10 11 11
- Input: 120 $\rightarrow$ Output: 10 110

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {  
    int n, temp, maxDigit = 0;  
    printf("Enter number: ");  
    scanf("%d", &n);
```

```
  
    int arr[10], k = 0;
```

```
  
    while(n > 0) {  
        temp = n;  
        int bd = 0, place = 1;  
        while(temp > 0) {  
            int d = temp % 10;  
            if(d > 0)
```

```
    bd += place;
        temp /= 10;
        place *= 10;
    }
    arr[k++] = bd;
    n -= bd;
}

printf("Minimum binary-decimals: ");
for(int i = 0; i < k; i++)
    printf("%d ", arr[i]);

return 0;
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class BinaryDecimalDecomposition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
        int[] arr = new int[10];
        int k = 0;
        while(n > 0) {
            int temp = n, bd = 0, place = 1;
            while(temp > 0) {
                int d = temp % 10;
```

```
if(d > 0)
    bd += place;
    temp /= 10;
    place *= 10;
}
arr[k++] = bd;
n -= bd;
}
```

```
System.out.print("Minimum binary-decimals: ");
for(int i = 0; i < k; i++)
    System.out.print(arr[i] + " ");
}
}
```

46) You're given a string as input. You have to reverse the string by keeping the punctuation and spaces. You must modify the source string itself without creating another string.

Example: Input: A man, in the boat says : I see 1-2-3 in the sky
Output: y kse, ht ni3 2lee slsy : a sta o-b-e ht nin amA

ANSWER IN C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int main() {
```

```
    char s[100];
```

```
    printf("Enter string: ");
```

```
    gets(s);
```

```
    int i = 0, j = strlen(s) - 1;
```

```
    while(i < j) {
```

```
        if(!isalnum(s[i]))
```

```
            i++;
```

```
        else if(!isalnum(s[j]))
```

```
            j--;
```

```
else {  
    char temp = s[i];  
    s[i] = s[j];  
    s[j] = temp;  
    i++;  
    j--;  
}  
}  
  
printf("Reversed string: %s", s);  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class ReverseKeepPunct {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter string: ");
        String str = sc.nextLine();
        char[] s = str.toCharArray();
        int i = 0, j = s.length - 1;
        while(i < j) {
            if(!Character.isLetterOrDigit(s[i]))
                i++;
            else if(!Character.isLetterOrDigit(s[j]))
                j--;
        }
    }
}
```



```
else {  
    char temp = s[i];  
    s[i] = s[j];  
    s[j] = temp;  
    i++;  
    j--;  
}  
}  
  
System.out.println("Reversed string: " + new String(s));  
}  
}
```

47) Given an array, find the minimum of all the greater numbers for each element in the array.

Sample Input:

Array: {2, 3, 7, 1, 8, 5, 11}

Expected Output:

{2>3, 3>5, 7>8, 1>2, 8>11, 5>7, 11}

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter size: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    for(i=0;i<n;i++) {
        int min = 1000000;
        for(j=i+1;j<n;j++) {
            if(a[j] > a[i] && a[j] < min)
                min = a[j];
        }
    }
```

```
if(min == 1000000)
    printf("%d ", a[i]);
else
    printf("%d>%d ", a[i], min);
}
return 0;
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class MinGreater {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] a = new int[n];

        for(int i=0;i<n;i++)

            a[i] = sc.nextInt();

        for(int i=0;i<n;i++) {

            int min = Integer.MAX_VALUE;
```

```
for(int j=i+1;j<n;j++) {  
    if(a[j] > a[i] && a[j] < min)  
        min = a[j];  
}  
if(min == Integer.MAX_VALUE)  
    System.out.print(a[i]+" ");  
else  
    System.out.print(a[i]+">" + min+" ");  
}  
}  
}
```

48)Find the largest sum contiguous subarray which should not have negative numbers.

Print the sum and the corresponding elements.

Sample Input:

Array: {2, 7, 5, 1, 3, 2, 9, 7}

Expected Output:

Sum: 14 Elements: 3, 2, 9

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n,i;
```

```
    printf("Enter size: ");
```

```
    scanf("%d",&n);
```

```
    int a[n];
```

```
    printf("Enter elements: ");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    int maxSum=0, sum=0, start=0, tempStart=0, end=0;
```

```
    for(i=0;i<n;i++) {
```

```
        if(a[i]<0) {
```



```
sum=0;
    tempStart=i+1;
} else {
    sum+=a[i];
    if(sum>maxSum) {
        maxSum=sum;
        start=tempStart;
        end=i;
    }
}
}
printf("Sum: %d\nElements: ", maxSum);
for(i=start;i<=end;i++)
    printf("%d ", a[i]);

return 0;
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class MaxPosSubarray {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        int n=sc.nextInt();

        int[] a=new int[n];

        for(int i=0;i<n;i++)

            a[i]=sc.nextInt();

        int maxSum=0, sum=0, start=0, tempStart=0, end=0;
```

```
for(int i=0;i<n;i++) {  
    if(a[i]<0) {  
        sum=0;  
        tempStart=i+1;  
    } else {  
        sum+=a[i];  
        if(sum>maxSum) {  
            maxSum=sum;  
            start=tempStart;  
            end=i;  
        }  
    }  
}
```

```
System.out.println("Sum: "+maxSum);
```

```
System.out.print("Elements: ");
```

```
for(int i=start;i<=end;i++)
```

```
    System.out.print(a[i]+" ");
```

```
}
```

```
}
```

49) Given an array of numbers and a window of size k,

Print the **maximum** of numbers inside the window for each step as the window moves from the beginning of the array.

Sample Input:

Enter the array size: 8 Enter the elements: 1, 3, 5, 2, 1, 8, 6, 9 Enter the window size: 3

Expected Output:

5 5 5 8 8 9

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {  
    int n,i,k,j,max;  
    printf("Enter array size: ");  
    scanf("%d",&n);  
    int a[n];  
    printf("Enter elements: ");  
    for(i=0;i<n;i++)  
        scanf("%d",&a[i]);  
    printf("Enter window size: ");  
    scanf("%d",&k);
```

```
for(i=0;i<=n-k;i++) {  
    max=a[i];  
    for(j=i+1;j<i+k;j++)  
        if(a[j]>max)  
            max=a[j];  
    printf("%d ",max);  
}  
  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class SlidingWindowMax {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter array size: ");
        int n=sc.nextInt();
        int[] a=new int[n];
        System.out.println("Enter elements: ");
        for(int i=0;i<n;i++)
            a[i]=sc.nextInt();
        System.out.println("Enter window size: ");
        int k=sc.nextInt();
```



```
for(int i=0;i<=n-k;i++) {  
    int max=a[i];  
    for(int j=i+1;j<i+k;j++)  
        if(a[j]>max)  
            max=a[j];  
    System.out.print(max+" ");  
}  
}
```

50) Find the minimum number of times required to represent a number as the sum of squares. You are to break a number down into the least number of perfect square terms (e.g., 1^2 , 2^2 , 3^2 , ...). Sample

Input: 12

Output: min: 3

ANSWER IN C

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int min(int a, int b) {  
    return a<b ? a : b;  
}
```

```
int main() {  
    int n,i,j;  
    printf("Enter number: ");  
    scanf("%d",&n);  
    int dp[n+1];  
    dp[0]=0;
```

```
for(i=1;i<=n;i++) {  
    dp[i]=i;  
    for(j=1;j*j<=i;j++)  
        dp[i]=min(dp[i],dp[i-j*j]+1);  
}  
  
printf("min: %d\n",dp[n]);  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class MinSquares {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        System.out.println("Enter number: ");
        int n=sc.nextInt();
        int[] dp=new int[n+1];
        dp[0]=0;
        for(int i=1;i<=n;i++) {
            dp[i]=i;
```

```
for(int j=1;j*j<=i;j++)  
    dp[i]=Math.min(dp[i],dp[i-j*j]+1);  
}  
  
System.out.println("min: "+dp[n]);  
}  
}
```

**51) Given an array of numbers,
Print the numbers without duplication.**

Sample Input:

Enter the array size: 4 Enter the elements: 1 1 2 4

Expected Output:

1 2 4

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {  
    int n,i,j,flag;  
    printf("Enter array size: ");  
    scanf("%d",&n);  
    int arr[n];  
    printf("Enter elements: ");  
    for(i=0;i<n;i++)  
        scanf("%d",&arr[i]);  
  
    printf("Output:\n");  
    for(i=0;i<n;i++) {  
        flag=0;
```



```
for(j=0;j<i;j++) {  
    if(arr[i]==arr[j]) {  
        flag=1;  
        break;  
    }  
}  
if(flag==0)  
    printf("%d ",arr[i]);  
}  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class RemoveDuplicates {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter array size: ");
        int n=sc.nextInt();
        int arr[]=new int[n];
        System.out.println("Enter elements: ");
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
        System.out.println("Output:");
```

```
for(int i=0;i<n;i++) {  
    boolean flag=false;  
    for(int j=0;j<i;j++) {  
        if(arr[i]==arr[j]) {  
            flag=true;  
            break;  
        }  
    }  
    if(!flag)  
        System.out.print(arr[i]+" ");  
}  
}
```

**52) Given an array of numbers and a number k,
Print the maximum possible k-digit number
which can be formed using the given numbers.**

Sample Input:

Enter the array size: 4 Enter the elements: 1 4 9 7 3
9 7 Enter number of digits: 3

Expected Output:

974

ANSWER IN C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int n,k,i;
```

```
    printf("Enter array size: ");
```

```
    scanf("%d",&n);
```

```
    char arr[n][20],max[20]="";
```

```
    printf("Enter elements: ");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%s",arr[i]);
```

```
    printf("Enter number of digits: ");
```

```
    scanf("%d",&k);
```

```
for(i=0;i<n;i++) {  
    if(strlen(arr[i])==k) {  
        if(strcmp(arr[i],max)>0)  
            strcpy(max,arr[i]);  
    }  
}  
  
printf("Output:\n%s",max[0]=='\0'?"No k-digit number found":max);  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class MaxKDigit {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter array size: ");
        int n=sc.nextInt();
        String arr[]=new String[n];
        System.out.println("Enter elements: ");
        for(int i=0;i<n;i++)
            arr[i]=sc.next();
        System.out.println("Enter number of digits: ");
        int k=sc.nextInt();
```

```
String max="";
for(int i=0;i<n;i++) {
    if(arr[i].length()==k) {
        if(max.equals("") || arr[i].compareTo(max)>0)
            max=arr[i];
    }
}
```

```
System.out.println("Output:");
if(max.equals(""))
    System.out.println("No k-digit number found");
else
    System.out.println(max);
}
}
```


53)Print the numbers which have the longest continuous range.

Sample Input 1:

Enter array size: 8 Enter array elements: 1 3 10 7 9 2 4
6

Expected Output 1:

1 2 3 4

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n,i,j;
```

```
    printf("Enter array size: ");
```

```
    scanf("%d",&n);
```

```
    int arr[n],visited[n],maxLen=0,start=0;
```

```
    printf("Enter array elements: ");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&arr[i]);
```

```
    for(i=0;i<n;i++)
```

```
        visited[i]=0;
```

```
for(i=0;i<n;i++) {  
    if(visited[i]) continue;  
    int len=1,cur=arr[i];  
    visited[i]=1;  
    j=cur-1;  
    while(1) {  
        int found=0;  
        for(int k=0;k<n;k++) {  
            if(arr[k]==j && !visited[k]) {  
                visited[k]=1;  
                len++;  
                j--;  
                found=1;  
                break;  
            }  
        }  
        if(!found) break;  
    }  
}
```

```
j=cur+1;
while(1) {
    int found=0;
    for(int k=0;k<n;k++) {
        if(arr[k]==j && !visited[k]) {
            visited[k]=1;
            len++;
            j++;
            found=1;
            break;
        }
    }
    if(!found) break;
}
if(len>maxLen) {
    maxLen=len;
    start=j-len; }
}
```

```
printf("Output:\n");  
    for(i=0;i<maxLen;i++)  
        printf("%d ",start+i);  
  
    return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;
public class LongestRange {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter array size: ");
        int n=sc.nextInt();
        int arr[]=new int[n];
        System.out.println("Enter array elements: ");
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
        HashSet<Integer> set=new HashSet<>();
        for(int i=0;i<n;i++)
            set.add(arr[i]);
```

```
int maxLen=0,start=0;
```

```
    for(int num:arr) {  
        if(!set.contains(num-1)) {  
            int curr=num,len=1;  
            while(set.contains(curr+1)) {  
                curr++;  
                len++;  
            }  
            if(len>maxLen) {  
                maxLen=len;  
                start=num;  
            }  
        }  
    }  
}
```

```
System.out.println("Output:");  
    for(int i=0;i<maxLen;i++)  
        System.out.print((start+i)+" ");  
    }  
}
```


54)Print the following pattern

Given a number n, print the first n rows of **Pascal's Triangle**.

Sample Input:

5

Expected Output:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n,i,j;
    printf("Enter n: ");
    scanf("%d",&n);
    int c;
    printf("Output:\n");
    for(i=0;i<n;i++) {
        c=1;
        for(j=0;j<=i;j++) {
            printf("%d ",c);
            c=c*(i-j)/(j+1);
        }
        return 0;
    }
```

ANSWER IN JAVA

```
import java.util.*;

public class PascalTriangle {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        System.out.println("Enter n: ");

        int n=sc.nextInt();

        System.out.println("Output:");

        for(int i=0;i<n;i++) {

            int c=1;

            for(int j=0;j<=i;j++) {
```

```
System.out.print(c+" ");
```

```
    c=c*(i-j)/(j+1);
```

```
    }
```

```
  }
```

```
}
```

```
}
```

55) Given an input array, find the number of occurrences of a particular number without looping (use hashing)

Variation:

Given an array of characters, print the characters that have 'n' number of occurrences.

If a character appears **consecutively**, it is **counted as 1 occurrence**.

Example Input:

a b a a b c c c d e

Explanation:

Here, a appears non-consecutively in two groups $\rightarrow$ count = 2

So, a has only **2 occurrences**.

ANSWER IN C

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100], prev=0;
    int i, n, hash[256]={0};
    printf("Enter number of characters: ");
    scanf("%d",&n);
    printf("Enter characters:\n");
    for(i=0;i<n;i++) {
        char c;
        scanf(" %c",&c);
```

```
if(c != prev) {  
    hash[c]++;  
    prev=c;  
}  
}
```

```
printf("Output:\n");  
for(i=0;i<256;i++) {  
    if(hash[i]>0)  
        printf("%c: %d\n",i,hash[i]);  
}  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;

public class NonConsecutiveOccurrence {

    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter number of characters: ");
        int n=sc.nextInt();
        char[] arr=new char[n];
        System.out.println("Enter characters:");
        for(int i=0;i<n;i++) {
            arr[i]=sc.next().charAt(0);
        }
    }
}
```



```
Map<Character,Integer> map=new HashMap<>();
    char prev=0;
    for(char c : arr) {
        if(c != prev) {
            map.put(c,map.getDefault(c,0)+1);
            prev=c;
        }
    }
    System.out.println("Output:");
    for(Map.Entry<Character,Integer> entry : map.entrySet()) {
        System.out.println(entry.getKey()+" "+entry.getValue());
    }
}
```

56)Find the maximum of three numbers

Example Input:

$a = 15, b = 22, c = 10$

Output:

Maximum is 22

ANSWER IN C

```
#include <stdio.h>

int main() {
    int a,b,c,max;
    printf("Enter three numbers: ");
    scanf("%d%d%d",&a,&b,&c);
    max = a;
    if(b > max) max = b;
    if(c > max) max = c;
    printf("Maximum is %d", max);
    return 0;
}
```

ANSWER IN JAVA

```
import java.util.*;
public class MaxOfThree {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter three numbers: ");
        int a=sc.nextInt();
        int b=sc.nextInt();
        int c=sc.nextInt();
        int max=a;
        if(b > max) max = b;
        if(c > max) max = c;
        System.out.println("Maximum is "+max);
    }
}
```

57)Print the total number of odd and even digits in the given number

Example Input:

1234567

Output:

ODD 4

EVEN 3

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, digit, odd=0, even=0;
    printf("Enter a number: ");
    scanf("%d",&n);
    while(n>0) {
        digit = n%10;
        if(digit%2==0)
            even++;
        else
```

```
odd++;
```

```
    n/=10;
```

```
}
```

```
printf("ODD %d\nEVEN %d", odd, even);
```

```
return 0;
```

```
}
```

ANSWER IN JAVA

```
import java.util.*;
public class OddEvenCount {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter a number: ");
        int n=sc.nextInt();
        int odd=0, even=0;
        while(n>0){
            int digit = n%10;
            if(digit%2==0)
                even++;
            else
```



```
odd++;
```

```
    n/=10;
```

```
}
```

```
System.out.println("ODD "+odd);
```

```
System.out.println("EVEN "+even);
```

```
}
```

```
}
```

58)Efficiently merging two sorted arrays with $O(1)$ extra space

ANSWER IN C

```
#include <stdio.h>
#include <math.h>
int nextGap(int gap) {
    if (gap <= 1) return 0;
    return (gap / 2) + (gap % 2);
}
int main() {
    int n, m;
    printf("Enter size of array1: ");
    scanf("%d",&n);
    int arr1[n];
    printf("Enter elements of array1: ");
    for(int i=0;i<n;i++)
        scanf("%d",&arr1[i]);
```

```
printf("Enter size of array2: ");
scanf("%d",&m);
int arr2[m];
printf("Enter elements of array2: ");
for(int i=0;i<m;i++)
    scanf("%d",&arr2[i]);
int gap = nextGap(n + m);
while (gap > 0) {
    int i, j;
    for (i = 0; i + gap < n; i++) {
        if (arr1[i] > arr1[i + gap]) {
            int temp = arr1[i];
            arr1[i] = arr1[i + gap];
            arr1[i + gap] = temp;
        }
    }
}
```

```
for (j = (gap > n) ? gap - n : 0; i < n && j < m; i++, j++) {  
    if (arr1[i] > arr2[j]) {  
        int temp = arr1[i];  
        arr1[i] = arr2[j];  
        arr2[j] = temp;  
    }  
}  
if (j < m) {  
    for (j = 0; j + gap < m; j++) {  
        if (arr2[j] > arr2[j + gap]) {  
            int temp = arr2[j];  
            arr2[j] = arr2[j + gap];  
            arr2[j + gap] = temp;  
        }  
    }  
}  
gap = nextGap(gap);  
}
```

```
printf("Merged arrays:\n");  
    for(int i=0;i<n;i++)  
        printf("%d ", arr1[i]);  
    for(int i=0;i<m;i++)  
        printf("%d ", arr2[i]);  
  
    return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;

public class MergeSortedArrays {
    public static int nextGap(int gap) {
        if (gap <= 1) return 0;
        return (gap / 2) + (gap % 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter size of array1: ");
        int n = sc.nextInt();
        int arr1[] = new int[n];
```

```
System.out.println("Enter elements of array1: ");  
    for(int i=0;i<n;i++)  
        arr1[i] = sc.nextInt();
```

```
System.out.println("Enter size of array2: ");  
int m = sc.nextInt();  
int arr2[] = new int[m];  
System.out.println("Enter elements of array2: ");  
for(int i=0;i<m;i++)  
    arr2[i] = sc.nextInt();
```

```
int gap = nextGap(n + m);  
while (gap > 0) {  
    int i, j;
```



```
for (i = 0; i + gap < n; i++) {  
    if (arr1[i] > arr1[i + gap]) {  
        int temp = arr1[i];  
        arr1[i] = arr1[i + gap];  
        arr1[i + gap] = temp;  
    }  
}
```

```
for (j = (gap > n) ? gap - n : 0; i < n && j < m; i++, j++) {  
    if (arr1[i] > arr2[j]) {  
        int temp = arr1[i];  
        arr1[i] = arr2[j];  
        arr2[j] = temp;  
    }  
}
```

```
if (j < m) {  
    for (j = 0; j + gap < m; j++) {  
        if (arr2[j] > arr2[j + gap]) {  
            int temp = arr2[j];  
            arr2[j] = arr2[j + gap];  
            arr2[j + gap] = temp;  
        }  
    }  
}  
gap = nextGap(gap);  
}  
System.out.println("Merged arrays:");  
for(int val: arr1)  
    System.out.print(val + " ");  
for(int val: arr2)  
    System.out.print(val + " ");  
}  
}
```

59)Insert an element at a particular index in an array

Example Input:

Array: [10, 20, 30, 40] Element to insert: 25

Index: 2

Output:

[10, 20, 25, 30, 40]

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, index, element;
```

```
    printf("Enter size of array: ");
```

```
    scanf("%d",&n);
```

```
    int arr[n+1];
```

```
    printf("Enter array elements: ");
```

```
    for(int i=0;i<n;i++)
```

```
        scanf("%d",&arr[i]);
```

```
    printf("Enter element to insert: ");
```

```
    scanf("%d",&element);
```

```
    printf("Enter index: ");
```

```
    scanf("%d",&index);
```

```
for(int i=n;i>index;i--)  
    arr[i]=arr[i-1];  
arr[index]=element;  
n++;  
printf("Array after insertion:\n");  
for(int i=0;i<n;i++) {  
    printf("%d",arr[i]);  
    if(i!=n-1) printf(", ");  
}  
printf("]");  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class InsertAtIndex {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter size of array:");
        int n = sc.nextInt();
        int arr[] = new int[n+1];
        System.out.println("Enter array elements:");
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
        System.out.println("Enter element to insert:");
        int element = sc.nextInt();
```

```
System.out.println("Enter index:");
```

```
int index = sc.nextInt();
```

```
for(int i=n;i>index;i--)
```

```
    arr[i]=arr[i-1];
```

```
arr[index]=element;
```

```
n++;
```

```
System.out.print("Array after insertion:\n[");
```

```
for(int i=0;i<n;i++) {
```

```
    System.out.print(arr[i]);
```

```
if(i!=n-1) System.out.print(", ");  
    }  
    System.out.print("]");  
}  
}
```


60) Given a large number, convert it to base 7

Example Input:

Number: 100

Output:

Base 7: 202

ANSWER IN C

```
#include <stdio.h>
int main() {
    int n, arr[100], i = 0;
    printf("Enter number: ");
    scanf("%d", &n);
    int num = n;
    if(n == 0) {
        printf("Base 7: 0\n");
        return 0;
    }
    while(n > 0) {
        arr[i++] = n % 7;
        n /= 7;
    }
```

```
printf("Base 7: ");  
    for(int j = i - 1; j >= 0; j--)  
        printf("%d", arr[j]);  
  
    return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class Base7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
        int num = n;
        StringBuilder sb = new StringBuilder();
        if(n == 0) {
            System.out.println("Base 7: 0");
            return;
        }
    }
}
```

```
while(n > 0) {  
    sb.insert(0, n % 7);  
    n /= 7;  
}  
  
System.out.println("Base 7: " + sb.toString());  
}  
}
```

61)Given an IP address, validate it based on the given conditions

Example Input:

IP Address: 192.168.1.1

Output:

Valid IP address

ANSWER IN C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int isValid(char *ip) {
    int num, dots = 0;
    char *ptr = strtok(ip, ".");
    while(ptr) {
        if(!isdigit(ptr[0]) && !(ptr[0]=='0' && ptr[1]=='\0'))
            return 0;
        num = atoi(ptr);
        if(num < 0 || num > 255)
            return 0;
```

```
ptr = strtok(NULL, ".");  
    dots++;  
}  
return dots == 4;  
}
```

```
int main() {  
    char ip[50];  
    printf("Enter IP Address: ");  
    scanf("%s", ip);  
  
    char temp[50];  
    strcpy(temp, ip);
```



```
if(isValid(temp))  
    printf("Valid IP address\n");  
else  
    printf("Invalid IP address\n");  
  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;
public class IPValidation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter IP Address: ");
        String ip = sc.next();
        String[] parts = ip.split("\\.");

        if(parts.length != 4) {
            System.out.println("Invalid IP address");
            return;
        }
    }
}
```

```
boolean valid = true;
    for(String part : parts) {
        try {
            int num = Integer.parseInt(part);
            if(num < 0 || num > 255) {
                valid = false;
                break;
            }
        } catch(Exception e) {
            valid = false;
            break;
        }
    }
}
```

```
if(valid)
    System.out.println("Valid IP address");
else
    System.out.println("Invalid IP address");
}
```

62)Sort parts of an array separately using peak values

Description:

A **peak value** is a number that is greater than its neighbors.
Use these as dividers and sort the sections between them.

Example Input:

Array: [10, 20, 30, 25, 40, 60, 20, 10, 70, 30]

Output:

[10, 20, 30, 25, 40, 60, 10, 20, 70, 30]

ANSWER IN C

```
#include <stdio.h>

void sort(int arr[], int l, int r) {
    for(int i = l; i < r; i++) {
        for(int j = i+1; j <= r; j++) {
            if(arr[i] > arr[j]) {
                int t = arr[i];
                arr[i] = arr[j];
                arr[j] = t;
            }
        }
    }
}

int main() {
    int n, arr[100];
    printf("Enter size: ");
    scanf("%d", &n);
```

```
printf("Enter elements: ");
```

```
for(int i = 0; i < n; i++)
```

```
    scanf("%d", &arr[i]);
```

```
int peaks[100], p=0;
```

```
for(int i=0;i<n;i++) {
```

```
    if((i==0 && arr[i]>arr[i+1]) ||
```

```
        (i==n-1 && arr[i]>arr[i-1]) ||
```

```
        (i>0 && i<n-1 && arr[i]>arr[i-1] && arr[i]>arr[i+1]))
```

```
        peaks[p++] = i;
```

```
}
```

```
int start=0;
```

```
for(int i=0;i<p;i++) {  
    sort(arr, start, peaks[i]);  
    start = peaks[i]+1;  
}  
if(start < n)  
    sort(arr, start, n-1);  
  
printf("Output:\n");  
for(int i=0;i<n;i++)  
    printf("%d ", arr[i]);  
  
return 0;  
}
```


ANSWER IN JAVA

```
import java.util.*;
public class PeakSort {
    static void sort(int[] arr, int l, int r) {
        for(int i=l;i<r;i++) {
            for(int j=i+1;j<=r;j++) {
                if(arr[i]>arr[j]) {
                    int t=arr[i];
                    arr[i]=arr[j];
                    arr[j]=t;
                }
            }
        }
    }
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("Enter size: ");  
    int n = sc.nextInt();  
    int[] arr = new int[n];  
    System.out.print("Enter elements: ");  
    for(int i=0;i<n;i++)  
        arr[i]=sc.nextInt();  
  
    ArrayList<Integer> peaks = new ArrayList<>();  
    for(int i=0;i<n;i++) {  
        if((i==0 && arr[i]>arr[i+1]) ||  
            (i==n-1 && arr[i]>arr[i-1]) ||  
            (i>0 && i<n-1 && arr[i]>arr[i-1] && arr[i]>arr[i+1]))  
            peaks.add(i);  
    }  
}
```

```
int start=0;
    for(int idx : peaks) {
        sort(arr, start, idx);
        start = idx+1;
    }
    if(start < n)
        sort(arr, start, n-1);

    System.out.println("Output:");
    for(int x : arr)
        System.out.print(x+" ");
    }
}
```

63)Insert 0 after consecutive (K times) of 1 is found

Example Input:

Number of bits: 12 Bits: 1 0 1 1 0 1 1 0 1 1 1 1

Consecutive K: 2

Output:

1 0 1 1 0 0 1 1 0 0 1 1 1 1 0

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, k, arr[100], res[200], i, j=0, count=0;
    printf("Enter number of bits: ");
    scanf("%d", &n);
    printf("Enter bits: ");
    for(i=0;i<n;i++)
        scanf("%d", &arr[i]);
    printf("Enter K: ");
    scanf("%d", &k);
    for(i=0;i<n;i++) {
        res[j++] = arr[i];
        if(arr[i]==1)
            count++;
    }
}
```

else

count=0;

if(count==k) {

res[j++] = 0;

count=0;

}

}

printf("Output:\n");

for(i=0;i<j;i++)

printf("%d ", res[i]);

return 0;

}

ANSWER IN JAVA

```
import java.util.*;

public class InsertZeroAfterKOnes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of bits: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter bits: ");
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
        System.out.print("Enter K: ");
        int k = sc.nextInt();
```

```
ArrayList<Integer> res = new ArrayList<>();
    int count=0;
    for(int i=0;i<n;i++) {
        res.add(arr[i]);
        if(arr[i]==1)
            count++;
        else
            count=0;
        if(count==k) {
            res.add(0);
            count=0;
        }
    }
}
```



```
System.out.println("Output:");  
    for(int x : res)  
        System.out.print(x+" ");  
    }  
}
```

64) Evaluate a given expression which has factorials and exponential terms

Example Input:

Expression: $3! + 2^3$

Output:

14 (Because $3! = 6$, $2^3 = 8$, so $6 + 8 = 14$)

ANSWER IN C

```
#include <stdio.h>
#include <string.h>
#include <math.h>
int fact(int n) {
    int f=1;
    for(int i=1;i<=n;i++)
        f*=i;
    return f;
}
int main() {
    char exp[100];
    printf("Enter expression: ");
    scanf("%[^\\n]", exp);
```

```
int res=0, i=0, num=0;
while(exp[i]!='\0') {
    if(exp[i]>='0' && exp[i]<='9') {
        num=0;
        while(exp[i]>='0' && exp[i]<='9') {
            num = num*10 + (exp[i]-'0');
            i++;
        }
        if(exp[i]=='!') {
            res += fact(num);
            i++;
        }
        else if(exp[i]=='^') {
            i++;
        }
    }
}
```

```
int p=0;
    while(exp[i]>='0' && exp[i]<='9') {
        p = p*10 + (exp[i]-'0');
        i++;
    }
    res += pow(num,p);
}
else
    i++;
}
printf("%d", res);
return 0;
}
```

ANSWER IN JAVA

```
import java.util.*;
public class EvalExpr {
    static int fact(int n) {
        int f=1;
        for(int i=1;i<=n;i++)
            f*=i;
        return f;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter expression: ");
        String exp = sc.nextLine();
        int res=0, i=0;
        while(i<exp.length()) {
```

```
if(Character.isDigit(exp.charAt(i))) {  
    int num=0;  
    while(i<exp.length() && Character.isDigit(exp.charAt(i))) {  
        num = num*10 + (exp.charAt(i)-'0');  
        i++;  
    }  
    if(i<exp.length() && exp.charAt(i)=='!') {  
        res += fact(num);  
        i++;  
    }  
    else if(i<exp.length() && exp.charAt(i)=='^') {  
        i++;  
        int p=0;
```

```
while(i<exp.length() && Character.isDigit(exp.charAt(i))) {  
    p = p*10 + (exp.charAt(i)-'0');  
    i++;  
}  
res += Math.pow(num,p);  
}  
}  
else  
    i++;  
}  
System.out.println(res);  
}  
}
```


65)Implement a Snake and Ladder game for a given 2D array of positions

Example Input:

Board Size: 5x5 Snakes: from 23 to 4 Ladders:
from 3 to 22 Start Position: 1

Output:

Minimum number of moves to reach 25: 3

ANSWER IN C

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 25
typedef struct Queue {
    int pos, moves;
} Queue;
int minMoves(int moves[]) {
    int visited[SIZE+1] = {0};
    Queue q[SIZE+1];
    int front=0, rear=0;
    q[rear].pos = 1;
    q[rear++].moves = 0;
    visited[1]=1;
```

```
while(front<rear) {  
    Queue curr = q[front++];  
    if(curr.pos == SIZE)  
        return curr.moves;  
    for(int dice=1; dice<=6; dice++) {  
        int next = curr.pos + dice;  
        if(next <= SIZE && !visited[next]) {  
            visited[next]=1;  
            if(moves[next]!=-1)  
                next = moves[next];  
            q[rear].pos = next;  
            q[rear++].moves = curr.moves+1;  
        }  
    }  
}
```

```
int main() {  
    int moves[SIZE+1];  
    for(int i=0;i<=SIZE;i++)  
        moves[i]=-1;  
    moves[3]=22; moves[23]=4;  
    printf("Minimum number of moves to reach 25: %d",  
minMoves(moves));  
    return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;
public class SnakeLadder {
    static class Node {
        int pos, moves;
        Node(int p, int m) { pos=p; moves=m; }
    }
    public static void main(String[] args) {
        int SIZE = 25;
        int[] moves = new int[SIZE+1];
        Arrays.fill(moves, -1);
        moves[3]=22;
        moves[23]=4;
```

```
boolean[] visited = new boolean[SIZE+1];
    Queue<Node> q = new LinkedList<>();
    q.add(new Node(1,0));
    visited[1]=true;

    while(!q.isEmpty()) {
        Node curr = q.poll();
        if(curr.pos == SIZE) {
            System.out.println("Minimum number of moves to reach 25: " +
curr.moves);
            return;
        }
        for(int dice=1; dice<=6; dice++) {
            int next = curr.pos + dice;
```

```
if(next<=SIZE && !visited[next]) {  
    visited[next]=true;  
    if(moves[next]!=-1)  
        next = moves[next];  
    q.add(new Node(next, curr.moves+1));  
}  
}  
}  
}  
}
```

66) Calculate the strength of the password string using predefined rules

Example Input:

Password: P@ssword123

Output:

Strength Score: 18 Strength: Strong

ANSWER IN C

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char pwd[100];
    printf("Enter password: ");
    scanf("%s", pwd);
    int len = strlen(pwd);
    int score = len * 4;
    int hasUpper=0, hasLower=0, hasDigit=0, hasSpecial=0;
```

```
for(int i=0;i<len;i++) {  
    if(isupper(pwd[i])) hasUpper=1;  
    else if(islower(pwd[i])) hasLower=1;  
    else if(isdigit(pwd[i])) { hasDigit=1; score+=4; }  
    else hasSpecial=1;  
}
```

```
if(hasUpper && hasLower) score+=5;  
if(hasSpecial) score+=5;
```

```
printf("Strength Score: %d\n", score);  
if(score>=15)  
    printf("Strength: Strong\n");
```

```
else if(score>=10)
    printf("Strength: Medium\n");
else
    printf("Strength: Weak\n");

return 0;
}
```

ANSWER IN JAVA

```
import java.util.*;

public class PasswordStrength {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter password: ");

        String pwd = sc.next();

        int score = pwd.length() * 4;

        boolean hasUpper=false, hasLower=false, hasDigit=false,
        hasSpecial=false;
```

```
for(char c : pwd.toCharArray()) {  
    if(Character.isUpperCase(c)) hasUpper=true;  
    else if(Character.isLowerCase(c)) hasLower=true;  
    else if(Character.isDigit(c)) { hasDigit=true; score+=4; }  
    else hasSpecial=true;  
}
```

```
if(hasUpper && hasLower) score+=5;  
if(hasSpecial) score+=5;
```

```
System.out.println("Strength Score: " + score);  
if(score>=15)  
    System.out.println("Strength: Strong");
```

```
else if(score>=10)
    System.out.println("Strength: Medium");
else
    System.out.println("Strength: Weak");
}
}
```

67) Given four points, determine if it's a square, rectangle, or other

Example Input:

Points: (0,0), (0,2), (2,2), (2,0)

Output:

Shape: Square

ANSWER IN C

```
#include <stdio.h>
#include <math.h>

int distSq(int x1, int y1, int x2, int y2) {
    return (x1-x2)*(x1-x2) + (y1-y2)*(y1-y2);
}

int main() {
    int x[4], y[4];
    printf("Enter 4 points:\n");
    for(int i=0;i<4;i++)
        scanf("%d %d", &x[i], &y[i]);
```



```
int d[6], k=0;
    for(int i=0;i<4;i++)
        for(int j=i+1;j<4;j++)
            d[k++] = distSq(x[i], y[i], x[j], y[j]);
for(int i=0;i<6;i++)
    for(int j=i+1;j<6;j++)
        if(d[i]>d[j]) { int t=d[i]; d[i]=d[j]; d[j]=t; }

if(d[0]==0) {
    printf("Shape: Other\n");
    return 0;
}
```

```
if(d[0]==d[1] && d[1]==d[2] && d[2]==d[3] && d[4]==d[5]) {  
    if(d[4]==2*d[0])  
        printf("Shape: Square\n");  
    else  
        printf("Shape: Other\n");  
}  
else if(d[0]==d[1] && d[2]==d[3] && d[4]==d[5] && d[0]==d[2] &&  
d[4]==d[0]+d[2])  
    printf("Shape: Rectangle\n");  
else  
    printf("Shape: Other\n");  
  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;
public class CheckShape {
    static int distSq(int x1, int y1, int x2, int y2) {
        return (x1-x2)*(x1-x2) + (y1-y2)*(y1-y2);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] x = new int[4], y = new int[4];
        System.out.println("Enter 4 points:");
        for(int i=0;i<4;i++) {
            x[i] = sc.nextInt();
            y[i] = sc.nextInt();
        }
    }
}
```

```
int[] d = new int[6];  
int k=0;  
for(int i=0;i<4;i++)  
    for(int j=i+1;j<4;j++)  
        d[k++] = distSq(x[i], y[i], x[j], y[j]);
```

```
Arrays.sort(d);
```

```
if(d[0]==0)  
    System.out.println("Shape: Other");  
else if(d[0]==d[1] && d[1]==d[2] && d[2]==d[3] && d[4]==d[5]) {  
    if(d[4]==2*d[0])  
        System.out.println("Shape: Square");
```

else

System.out.println("Shape: Other");

}

else if(d[0]==d[1] && d[2]==d[3] && d[4]==d[5] && d[0]==d[2] &&
d[4]==d[0]+d[2])

System.out.println("Shape: Rectangle");

else

System.out.println("Shape: Other");

}

}

68)Print the second most frequently occurring number in a given series

Example Input:

1 1 2 3 1 2 4

Output:

2

ANSWER IN C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, arr[100], freq[100] = {0};
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements: ");
```

```
    for(int i=0;i<n;i++) {
```

```
        scanf("%d", &arr[i]);
```

```
        freq[arr[i]]++;
```

```
    }
```

```
    int first = 0, second = 0;
```

```
    for(int i=0;i<100;i++) {
```

```
if(freq[i]>freq[first]) {  
    second = first;  
    first = i;  
}  
else if(freq[i]>freq[second] && i!=first)  
    second = i;  
}  
  
printf("Second most frequent: %d\n", second);  
return 0;  
}
```


ANSWER IN JAVA

```
import java.util.*;

public class SecondMostFrequent {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<Integer, Integer> map = new HashMap<>();
        for(int i=0;i<n;i++) {
            int num = sc.nextInt();
            map.put(num, map.getDefault(num, 0)+1);
        }
        int firstFreq=0, secondFreq=0, firstNum=0, secondNum=0;
        for(Map.Entry<Integer,Integer> e : map.entrySet()) {
```

```
if(e.getValue()>firstFreq) {
    secondFreq = firstFreq;
    secondNum = firstNum;
    firstFreq = e.getValue();
    firstNum = e.getKey();
}
else if(e.getValue()>secondFreq && e.getKey() != firstNum) {
    secondFreq = e.getValue();
    secondNum = e.getKey();
}
}

System.out.println("Second most frequent: " + secondNum);
}
}
```

69)Print only numbers which are present in the Fibonacci series

(Fibonacci series: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...)

Example 1:

Input: 2 10 4 8 Output: 2 8

Example 2:

Input: 1 10 6 8 13 21 Output: 1 8 13 21

ANSWER IN C

```
#include <stdio.h>
#include <math.h>
int isFibo(int n) {
    int x = 5*n*n + 4;
    int y = 5*n*n - 4;
    int s1 = sqrt(x);
    int s2 = sqrt(y);
    return (s1*s1==x || s2*s2==y);
}
int main() {
    int n, arr[100];
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements: ");
```

```
for(int i=0;i<n;i++)  
    scanf("%d",&arr[i]);  
  
printf("Fibonacci numbers: ");  
for(int i=0;i<n;i++) {  
    if(isFibo(arr[i]))  
        printf("%d ",arr[i]);  
}  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.*;
public class FibCheck {
    static boolean isFibo(int n) {
        int x = 5*n*n + 4;
        int y = 5*n*n - 4;
        int s1 = (int)Math.sqrt(x);
        int s2 = (int)Math.sqrt(y);
        return s1*s1==x || s2*s2==y;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```
int[] arr = new int[n];  
    for(int i=0;i<n;i++)  
        arr[i]=sc.nextInt();
```

```
System.out.print("Fibonacci numbers: ");
```

```
for(int i=0;i<n;i++) {
```

```
    if(isFibo(arr[i]))
```

```
        System.out.print(arr[i]+" ");
```

```
    }
```

```
}
```

```
}
```

70)Print pattern like this

Example Input:

1

Output:

0

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    for(int i=0;i<n;i++)
        printf("0\n");

    return 0;
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class PatternZero {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for(int i=0;i<n;i++)

            System.out.println("0");

    }

}
```

71)Prime number – Print first n prime numbers

Example Input:

$n = 5$

Output:

2 3 5 7 11

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n, count = 0, num = 2;
    scanf("%d", &n);
    while(count < n) {
        int isPrime = 1;
        for(int i = 2; i*i <= num; i++) {
            if(num % i == 0) {
                isPrime = 0;
                break;
            }
        }
    }
}
```

```
if(isPrime) {  
    printf("%d ", num);  
    count++;  
}  
num++;  
}  
  
return 0;  
}
```

ANSWER IN JAVA

```
import java.util.Scanner;

public class FirstNPrimes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int count = 0, num = 2;

        while(count < n) {
            boolean isPrime = true;
            for(int i = 2; i*i <= num; i++) {
```

```
if(num % i == 0) {  
    isPrime = false;  
    break;  
}  
}  
if(isPrime) {  
    System.out.print(num + " ");  
    count++;  
}  
num++;  
}  
}  
}
```

72)Find the extra element and its index

Example 1:

Input: Array 1 = [10, 20, 30, 12, 5] Array 2 = [10, 5, 30, 20]

Output: 12 is the extra element in array 1 at index 4

ANSWER IN C

```
#include <stdio.h>

int main() {
    int n1, n2, arr1[100], arr2[100], found;
    scanf("%d", &n1);
    for(int i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);
    scanf("%d", &n2);
    for(int i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);
    for(int i = 0; i < n1; i++) {
        found = 0;
```